

GUIA DIDACTICA DE ACTIVIDADES FORMATIVAS

API REST y Frontend para la coleccion orders

Servicio Nacional de Aprendizaje - SENA
Centro de Electricidad, Electrónica y Telecomunicaciones
Programa: Análisis y Desarrollo de Software

Propósito: esta guía los acompañara en la construcción integral de una solución backend y frontend a partir de una colección de pedidos. A lo largo de las actividades aplicaran principios de arquitectura de software, control de versiones, documentación técnica, despliegue en la nube y sustentación oral de su trabajo.

1. Modelo base de datos y dominio

El dominio de trabajo corresponde a un escenario comercial conformado por cinco entidades: clientes (Customer), pedidos (Order), items de pedido (OrderItem), productos (Product) y proveedores (Supplier). La entidad Order es el agregado principal de la API: a partir de ella se acceden los demás recursos relacionados.

Customer 1 --- N Order 1 --- N OrderItem N --- 1 Product N --- 1 Supplier

Entidad	Campos principales
Customer	id, firstName, lastName, city, country, phone
Order	id, orderDate, orderNumber, customerId, totalAmount
OrderItem	id, orderId, productId, unitPrice, quantity
Product	id, productName, supplierId, unitPrice, package, isDiscontinued
Supplier	id, companyName, contactName, contactTitle, city, country, phone, fax

2. Contrato mínimo obligatorio de endpoints

Cada equipo debe implementar, como mínimo, todos los endpoints marcados como Obligatorios en la siguiente tabla. Los endpoints Complementarios son opcionales: permiten enriquecer la solución y ofrecer un frontend mas completo, por lo que se valoran positivamente en la evaluación. Importante: los métodos HTTP validos son GET, POST, PUT, PATCH y DELETE; no existe un método llamado "PUTCH".

Tipo	Metodo	Endpoint	Funcion	Entrada principal	Respuesta esperada
Obligatorio	GET	/api/v1/orders	Listar pedidos con paginacion, filtros y ordenamiento	Query: page, limit, customerId, dateFrom, dateTo, sort	200 OK: lista paginada de orders
Obligatorio	GET	/api/v1/orders/{orderId}	Consultar el detalle de un pedido, incluyendo cliente e items	Path: orderId	200 OK / 404 Not Found
Obligatorio	POST	/api/v1/orders	Crear un pedido a partir de un cliente y una lista de productos	Body: customerId, items[{productId, quantity}]	201 Created: order creado
Obligatorio	PUT	/api/v1/orders/{orderId}	Reemplazar completamente un pedido, incluidos sus items	Path: orderId. Body completo del pedido	200 OK / 400 Bad Request / 404
Obligatorio	PATCH	/api/v1/orders/{orderId}	Actualizar parcialmente datos del pedido	Path: orderId. Body parcial: orderDate, customerId, status opcional	200 OK / 404
Obligatorio	DELETE	/api/v1/orders/{orderId}	Eliminar o anular un pedido segun regla de negocio definida	Path: orderId	204 No Content / 404
Obligatorio	GET	/api/v1/orders/{orderId}/items	Listar los items de un pedido	Path: orderId	200 OK: lista de orderItems
Obligatorio	POST	/api/v1/orders/{orderId}/items	Agregar un producto a un	Body: productId, quantity	201 Created; recalcula

			pedido existente		totalAmount
Obligatorio	PATCH	/api/v1/orders/{orderId}/items/{itemId}	Actualizar cantidad o precio unitario de un item	Body parcial: quantity, unitPrice	200 OK; recalcula totalAmount
Obligatorio	DELETE	/api/v1/orders/{orderId}/items/{itemId}	Eliminar un item de un pedido	Path: orderId, itemId	204 No Content; recalcula totalAmount
Complementario	GET	/api/v1/customers	Listar clientes	Query: page, limit, country, city, search	200 OK
Complementario	GET	/api/v1/customers/{customerId}	Consultar detalle de cliente	Path: customerId	200 OK / 404
Complementario	GET	/api/v1/customers/{customerId}/orders	Listar pedidos asociados a un cliente	Path: customerId	200 OK
Complementario	POST	/api/v1/customers	Crear cliente	Body: firstName, lastName, city, country, phone	201 Created
Complementario	PATCH	/api/v1/customers/{customerId}	Actualizar parcialmente un cliente	Body parcial: city, country, phone, etc.	200 OK
Obligatorio	GET	/api/v1/products	Listar productos	Query: page, limit, supplierId, search, discontinued	200 OK
Obligatorio	GET	/api/v1/products/{productId}	Consultar detalle de producto con proveedor	Path: productId	200 OK / 404
Complementario	POST	/api/v1/products	Crear producto	Body: productName, supplierId, unitPrice, package, isDiscontinued	201 Created
Complementario	PUT	/api/v1/products/{productId}	Reemplazar completamente producto	Body completo del producto	200 OK
Complementario	PATCH	/api/v1/products/{productId}	Actualizar parcialmente producto, por ejemplo precio o estado	Body parcial: unitPrice, isDiscontinued	200 OK
Complementario	DELETE	/api/v1/products/{productId}	Eliminar producto si no tiene dependencias, o marcarlo como discontinuado	Path: productId	204 No Content o 409 Conflict
Complementario	GET	/api/v1/suppliers	Listar proveedores	Query: page, limit, country, search	200 OK
Complementario	GET	/api/v1/suppliers/{supplierId}	Consultar proveedor	Path: supplierId	200 OK / 404
Complementario	GET	/api/v1/suppliers/{supplierId}/products	Listar productos de un proveedor	Path: supplierId	200 OK
Complementario	POST	/api/v1/suppliers	Crear proveedor	Body: companyName, contactName, contactTitle, city, country, phone, fax	201 Created
Complementario	PATCH	/api/v1/suppliers/{supplierId}	Actualizar parcialmente proveedor	Body parcial	200 OK
Obligatorio	GET	/api/v1/health	Verificar disponibilidad del servicio	Sin parametros	200 OK: {status:"ok"}
Obligatorio	GET	/api/v1/docs	Acceder a la documentacion OpenAPI/Swagger	Sin parametros	200 OK: UI/documentacion

3. Reglas funcionales obligatorias de la API

- Calculen el campo totalAmount de cada pedido como la sumatoria de unitPrice multiplicado por quantity de todos sus items.

- Antes de crear un pedido, validen que el customerId exista en la base de datos y que cada productId enviado tambien exista; en caso contrario, devuelvan 400 o 404 segun corresponda.
- Cada vez que se agregue, modifique o elimine un item, el totalAmount del pedido debe recalcularse automaticamente.
- Mantengan respuestas de error consistentes en toda la API, usando los codigos 400 (datos invalidos), 404 (recurso no encontrado), 409 (conflicto) y 500 (error interno), siempre acompanados de un mensaje descriptivo.
- Expongan la documentacion de la API mediante OpenAPI/Swagger y publiquen un endpoint de salud que permita verificar que el servicio esta activo.
- Versionen todas las rutas bajo el prefijo /api/v1 para facilitar futuras evoluciones de la API.

4. Actividad 1: Desarrollo de API REST

Objetivo general: en esta actividad disenaran, construiran, documentaran, probaran y desplegaran una API REST sobre la coleccion orders. Durante el proceso aplicaran buenas practicas de arquitectura por capas, el flujo de trabajo Gitflow, commits semanticos (Conventional Commits) y versionamiento semantico (SemVer).

4.1 Distribucion de tecnologias

Grupo	Stack obligatorio	Observaciones
A	Python + Uvicorn + ambiente virtual env	Se recomienda FastAPI por su integracion natural con OpenAPI.
B	Node.js + Express	Se recomienda separar rutas, controladores, servicios y repositorios.

4.2 Endpoints que deben construir

Para aprobar la actividad deben entregar como minimo: el CRUD completo de orders, la gestion de items dentro de cada pedido, las consultas basicas de products (listar y detalle) y los endpoints de health y docs. Implementar adicionalmente customers y suppliers no es obligatorio, pero eleva la calidad funcional de la solucion y habilita un frontend mas robusto en la Actividad 2.

Metodo	Endpoint	Debe permitir
GET	/api/v1/orders	Listar pedidos con paginacion, filtros y ordenamiento
GET	/api/v1/orders/{orderId}	Consultar el detalle de un pedido, incluyendo cliente e items
POST	/api/v1/orders	Crear un pedido a partir de un cliente y una lista de productos
PUT	/api/v1/orders/{orderId}	Reemplazar completamente un pedido, incluidos sus items
PATCH	/api/v1/orders/{orderId}	Actualizar parcialmente datos del pedido
DELETE	/api/v1/orders/{orderId}	Eliminar o anular un pedido segun regla de negocio definida
GET	/api/v1/orders/{orderId}/items	Listar los items de un pedido
POST	/api/v1/orders/{orderId}/items	Agregar un producto a un pedido existente
PATCH	/api/v1/orders/{orderId}/items/{itemId}	Actualizar cantidad o precio unitario de un item
DELETE	/api/v1/orders/{orderId}/items/{itemId}	Eliminar un item de un pedido
GET	/api/v1/products	Listar productos
GET	/api/v1/products/{productId}	Consultar detalle de producto con proveedor
GET	/api/v1/health	Verificar disponibilidad del servicio
GET	/api/v1/docs	Acceder a la documentacion OpenAPI/Swagger

4.3 Ejemplos de payloads obligatorios

POST /api/v1/orders

```
{
  "customerId": 1,
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 3, "quantity": 1 }
  ]
}
```

PATCH /api/v1/orders/1/items/2

```
{
  "quantity": 5
}
```

```
PATCH /api/v1/products/4
{
  "unitPrice": 19.50,
  "isDiscontinued": false
}
```

4.4 Arquitectura sugerida backend

```
HTTP Client -> Routes/Controllers -> Services -> Repositories/Data Access -> Data Source
|-> Validators/Schemas
|-> Error Handler
|-> OpenAPI Docs
```

4.5 Entregables Actividad 1

- Repositorio GitHub con Gitflow visible en ramas main, develop y feature/*.
- Commits semanticos: feat, fix, docs, test, refactor, chore.
- Versionamiento semantico: v1.0.0 como primera version estable.
- API desplegada y URL funcional.
- Documentacion Swagger/OpenAPI.
- Informe tecnico con arquitectura, decisiones, hitos, pruebas y evidencias.
- Presentacion y sustentacion tecnica.

5. Actividad 2: Desarrollo frontend para consumir la API

Objetivo general: en esta actividad construiran una interfaz web responsiva utilizando Next.js y Tailwind CSS, que consuma los endpoints desarrollados en la Actividad 1. La aplicacion debe permitir al usuario final consultar, crear, editar y eliminar informacion del dominio orders, ofreciendo una experiencia clara tanto en escritorio como en dispositivos moviles.

5.1 Pantallas minimas obligatorias

Pantalla	Endpoints consumidos	Funcion esperada
Dashboard	GET /api/v1/orders, GET /api/v1/products	Resumen de pedidos, total vendido, productos activos y ultimos pedidos.
Listado de pedidos	GET /api/v1/orders	Tabla con busqueda, filtros por cliente/fecha y paginacion.
Detalle de pedido	GET /api/v1/orders/{orderId}, GET /api/v1/orders/{orderId}/items	Visualizar cliente, items, productos y total.
Crear pedido	POST /api/v1/orders, GET /api/v1/customers, GET /api/v1/products	Formulario para seleccionar cliente y productos.
Editar pedido	PATCH /api/v1/orders/{orderId}, PATCH /api/v1/orders/{orderId}/items/{itemId}	Modificar fecha, cliente o cantidades.
Gestion de productos	GET/POST/PATCH/DELETE /api/v1/products	CRUD de productos y estado discontinuado.
Gestion de clientes	GET/POST/PATCH /api/v1/customers	Administracion basica de clientes.
Estado del sistema	GET /api/v1/health, GET /api/v1/docs	Verificar API y enlazar documentacion.

5.2 Arquitectura sugerida frontend

```
Next.js App Router -> Pages/Routes -> Components -> Services/API Client -> API REST
|-> Hooks
|-> Types/Interfaces
|-> Tailwind UI
```

5.3 Entregables Actividad 2

- Aplicacion Next.js + Tailwind desplegada.
- Consumo real de endpoints de la Actividad 1.
- Diseno responsivo para web y movil.
- Repositorio GitHub con Gitflow y commits semanticos.
- Informe tecnico con arquitectura frontend, componentes, integracion API y evidencias.
- Presentacion y sustentacion con demo funcional.

6. Pruebas minimas esperadas

Caso	Endpoint	Resultado esperado
Crear pedido valido	POST /api/v1/orders	201 Created y total calculado.
Consultar pedido inexistente	GET /api/v1/orders/9999	404 Not Found.
Agregar item a pedido	POST /api/v1/orders/{id}/items	201 Created y total recalculado.
Actualizar cantidad	PATCH /api/v1/orders/{id}/items/{itemId}	200 OK y total recalculado.
Eliminar item	DELETE /api/v1/orders/{id}/items/{itemId}	204 No Content.
Listar con paginacion	GET /api/v1/orders?page=1&limit=10	200 OK con metadata de paginacion.
Validar API viva	GET /api/v1/health	200 OK.
Acceder documentacion	GET /api/v1/docs	Documentacion disponible.

7. Rubrica sintetica de evaluacion

Criterio	Peso	Evidencia
Funcionamiento backend/API	25%	Endpoints obligatorios, validaciones y codigos HTTP correctos.
Arquitectura y buenas practicas	20%	Separacion por capas, manejo de errores, estructura limpia.
Frontend e integracion	20%	Pantallas funcionales, consumo real de API, responsividad.
Git, Gitflow y versionamiento	15%	Ramas, commits semanticos, tags SemVer.
Documentacion e informe tecnico	10%	Swagger, README, informe con evidencias.
Sustentacion y demo	10%	Explicacion tecnica, pruebas y defensa de decisiones.

8. Criterios de calidad institucional

- Todo repositorio entregado debe incluir un archivo README con la descripcion del proyecto y las instrucciones para clonarlo, instalarlo y ejecutarlo localmente; sin este documento la entrega no sera recibida.
- No se aceptara un frontend que utilice datos quemados (hardcoded) cuando la API de la Actividad 1 ya este desplegada y operativa: la aplicacion debe consumir endpoints reales.
- Durante la sustentacion, cada equipo debera demostrar en vivo al menos un flujo completo, que incluya: listar pedidos, crear un pedido nuevo, editar un item y verificar que el total se recalcula correctamente.
- La sustentacion debe evidenciar el repositorio en GitHub, el despliegue en la nube, la documentacion Swagger/OpenAPI y la ejecucion de pruebas en tiempo real.